

# **YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT**

**Project – High Level Design (HLD)**

**on**

**Identity Security Framework for Supply Chain**

**Student Name: Alvia Dsouza**

**Industry Mentor:  
Mr.Shashank**

**Guided by: Ms. Divya N Shetty**

## Table of Contents

| <b>Sl.no</b> | <b>Contents</b>                     | <b>Page</b> |
|--------------|-------------------------------------|-------------|
| <b>1</b>     | <b>Introduction</b>                 | 1-2         |
| 1.1          | Scope of the document               |             |
| 1.2          | Intended Audience                   |             |
| 1.3          | System overview                     |             |
| <b>2</b>     | <b>System Design</b>                | 2-8         |
| 2.1          | Application Design                  |             |
| 2.2          | Process Flow                        |             |
| 2.3          | Information Flow                    |             |
| 2.4          | Components Design                   |             |
| 2.5          | Key Design Considerations           |             |
| 2.6          | API Catalogue                       |             |
| <b>3</b>     | <b>Data Design</b>                  | 8-10        |
| 3.1          | Data Model                          |             |
| 3.2          | Data Access Mechanism               |             |
| 3.3          | Data Retention Policies             |             |
| 3.4          | Data Migration                      |             |
| <b>4</b>     | <b>Interfaces</b>                   | 11-12       |
| <b>5</b>     | <b>State and Session Management</b> | 12          |
| <b>6</b>     | <b>Caching</b>                      | 13          |
| <b>7</b>     | <b>Non-Functional Requirements</b>  | 13-14       |
| 7.1          | Security Aspects                    |             |
| 7.2          | Performance Aspects                 |             |
| <b>8</b>     | <b>References</b>                   | 15          |

# **1. Introduction**

## **1.1 Scope of the document**

This document covers the high-level architecture of the Identity Security Framework for Supply Chain web application. It describes the overall system structure and three-layer architecture, the major functional modules and their responsibilities, the technology stack and the rationale behind each technology choice, the flow of data between the user's browser, the Flask application server, and the SQLite database, the database design at the entity level, the user interface structure and navigation flow, and the security and performance considerations of the system.

This document does not cover specific function implementations, line-by-line code logic, exact SQL schema definitions, or detailed validation rules— those are documented separately in the Low Level Design document. The scope is also limited to the identity and access management layer of the supply chain system and does not cover supply chain business operations such as inventory management, order tracking, or logistics.

## **1.2 Intended Audience**

This document is intended for academic guides, project evaluators, industry mentors, and any stakeholder who needs to understand the overall design, architecture, and purpose of the system without requiring deep technical knowledge of Flask or Python. A general understanding of web applications, databases, and basic software architecture concepts is sufficient to follow this document.

## **1.3 System overview**

The Identity Security Framework for Supply Chain is a multi-page web application that provides a complete identity and access management solution for a supply chain environment.

The system allows three categories of users: Administrators, Suppliers, and Distributors - to register with the platform, log in securely using their credentials, and access role-appropriate content after authentication.

The core security features of the system are as follows. All passwords are hashed before storage so that plain text passwords never exist in the database. User sessions are managed through cryptographically signed cookies that cannot be forged or tampered with. All sensitive pages are protected from unauthenticated access through Flask-Login's session guard. Access to administrative functionality is strictly restricted to users with the Admin role through server-side role checks.

The system is built on a standard three-layer web application architecture consisting of a presentation layer built with HTML and CSS, an application layer built with Flask and Python, and a data layer using SQLite accessed through Flask-SQLAlchemy. It is designed for local development and academic demonstration, with a clear path to production-grade enhancement.

## **2. System Design**

### **2.1 Application Design**

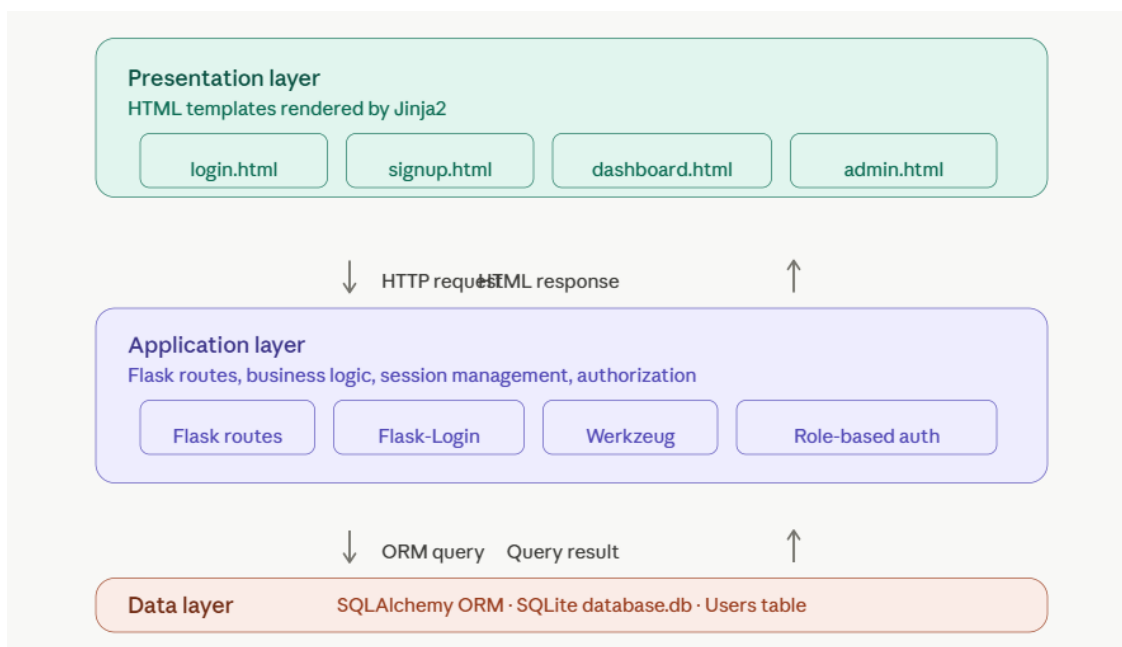
The system follows a three-layer architecture where each layer has a clearly defined responsibility.

The Presentation Layer is the topmost layer and consists of all HTML templates rendered by the Jinja2 templating engine. It includes four pages — the login page, the signup page, the dashboard, and the admin panel. Data is submitted from this layer to the application layer via HTTP POST requests through HTML forms, and responses are received as rendered HTML pages or HTTP redirects.

The Application Layer is implemented as a Flask web application running on Python 3.x. This layer contains all business logic including form input processing, server-side validation, password hashing, session management, role-based authorization, and database interaction. It acts as the bridge between the user-facing templates and the underlying database.

The Data Layer consists of an SQLite relational database accessed through Flask-SQLAlchemy. This layer handles the persistent storage of all user records and receives queries from the application layer through the ORM.

The application is organized into two backend Python files and a templates directory. `models.py` defines the User data model. `app.py` contains the Flask app instance, all configuration settings, and all route handler functions. The templates directory contains the four HTML pages.



## 2.2 Process Flow

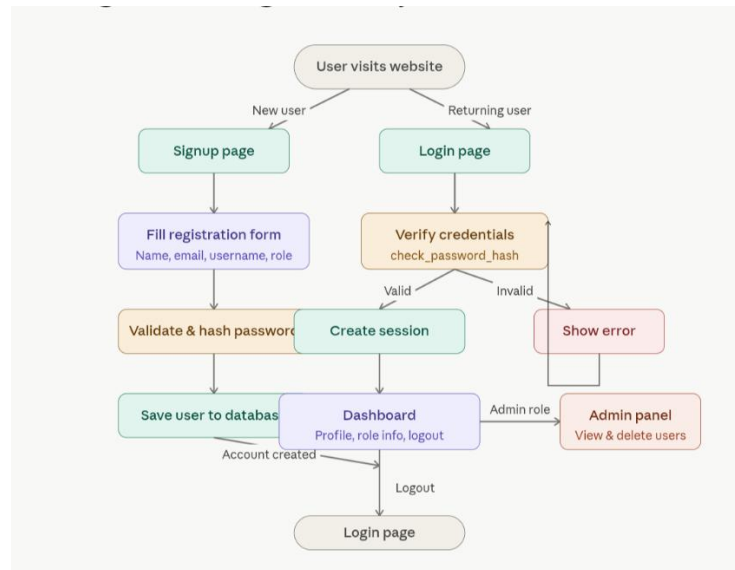
At a high level the system operates through four main user journeys:

**New User Registration Journey:** A new user navigates to the signup page and fills in the registration form with their first name, last name, email, username, password, and selected role. The system validates all inputs server-side, checks that the username and email are not already taken, hashes the password, and saves the new user to the database. The user is then directed to the login page with a success message.

**Returning User Login Journey:** An existing user submits their username and password on the login page. The system retrieves the user record from the database, verifies the password against the stored hash, and on success creates a signed session and redirects to the dashboard. If credentials are invalid, the same generic error message is shown regardless of whether the username or password was wrong, to prevent username enumeration.

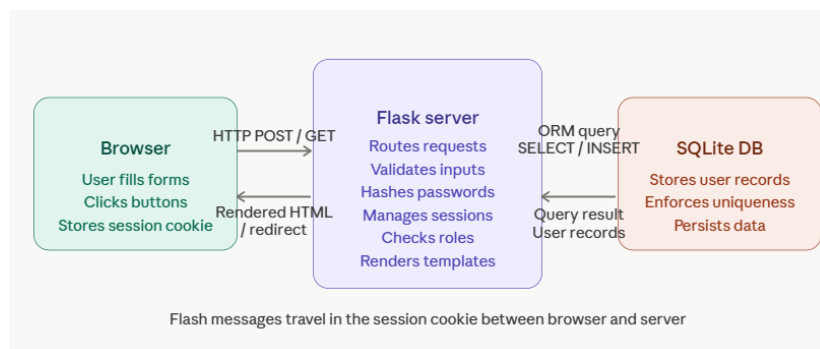
**Authenticated User Session Journey:** After login the user navigates to their dashboard which displays their profile information and role. The session persists across page navigations through the signed cookie managed by Flask-Login. When the user logs out the session is cleared and they are returned to the login page.

**Admin Management Journey:** An authenticated admin user can view a complete list of all registered users with their usernames, emails, and roles. The admin can delete any user except their own account, which is protected by a self-deletion guard. Non-admin users who attempt to access the admin panel directly are denied by the server-side role checks.



## 2.3 Information Flow

The user interacts with the application through the browser by filling forms and clicking buttons. Submissions are sent as HTTP POST requests and page navigations as HTTP GET requests to the Flask server. The server maps each request to the appropriate route handler, which validates input, interacts with the database through SQLAlchemy, and returns either a rendered HTML page or a redirect. Flash messages are stored in the session and displayed on the next page the user sees.



## 2.4 Components Design

The system has five major components:

**Registration Component :** Collects user details through the signup form, enforces a 7-step sequential server-side validation sequence, hashes the password using Werkzeug's `generate_password_hash()` before any database write, and persists the new user record. It is the only component that creates records in the database.

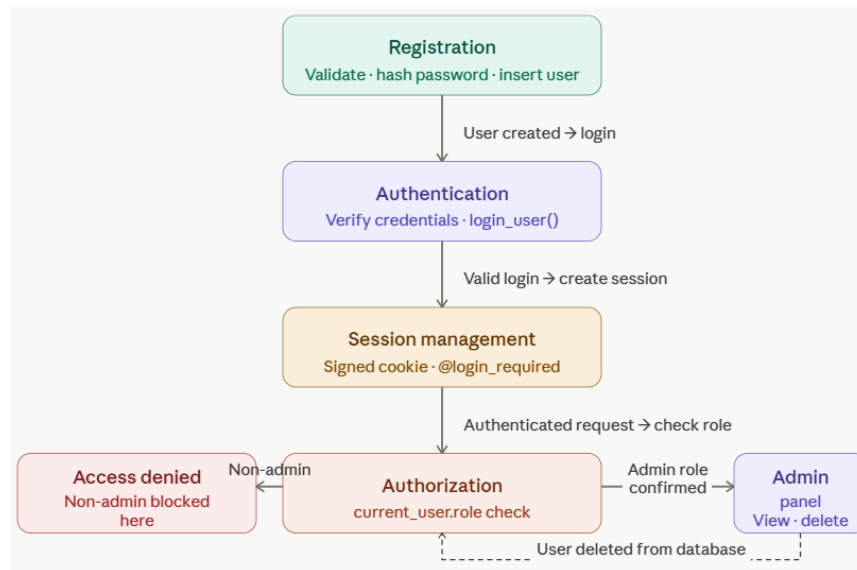
**Authentication Component :** Retrieves the user record from the database based on the submitted username and uses Werkzeug's `check_password_hash()` to verify the password in a timing-safe manner. On successful verification it calls Flask-Login's `login_user()` to initiate a signed session cookie.

**Session Management Component :** Maintains the user's authenticated state across multiple HTTP requests using Flask-Login's signed session cookies. The `@login_required` decorator is applied to all protected routes, automatically redirecting unauthenticated users to the login page while preserving the originally requested URL.

**Authorization Component :** Enforces role-based access control by checking `current_user.role` on every request to a restricted resource. Admin users are permitted to access the admin panel and delete functionality. Supplier and Distributor users are restricted to the dashboard and receive an Access Denied response if they attempt to access admin routes directly.

**Admin Management Component :** Provides administrative functionality exclusively to Admin users. It allows viewing a complete list of all registered users and permanently deleting accounts, subject to the constraint that an admin cannot delete their own account, preventing accidental loss of admin access to the system.





## 2.5 Key Design Considerations

Security first — Passwords are never stored in plain text. Session cookies are cryptographically signed. Access control is enforced entirely on the server side.

Separation of authentication and authorization — Login and session handling are managed separately from role checks, keeping both concerns clean and independently testable.

Role-based access control — The three-role model of Admin, Supplier, and Distributor reflects real supply chain organizational structures and makes the system directly applicable to real scenarios.

Username enumeration prevention — The login route returns the same error message whether the username does not exist or the password is wrong, preventing attackers from discovering registered usernames.

Standard technologies — Every library used is widely adopted and production-proven, making the skills learned directly transferable to professional development.

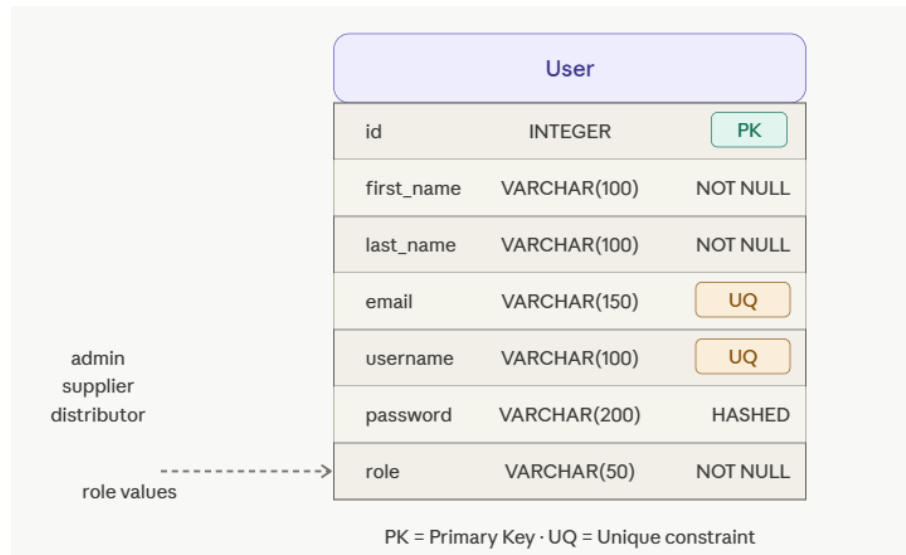
## 2.6 API Catalogue

| Route        | Method    | Auth Required | Role  | Description         |
|--------------|-----------|---------------|-------|---------------------|
| /            | GET       | No            | None  | Redirects to /login |
| /signup      | GET, POST | No            | None  | User registration   |
| /login       | GET, POST | No            | None  | User login          |
| /dashboard   | GET       | Yes           | Any   | User dashboard      |
| /logout      | GET       | Yes           | Any   | Clears session      |
| /admin       | GET       | Yes           | Admin | View all users      |
| /delete/<id> | GET       | Yes           | Admin | Delete a user       |

## 3. Data Design

### 3.1 Data Model

The system has a single data entity — the User. It contains a unique numeric ID, first name, last name, email (unique), username (unique), hashed password, and an assigned role. The role field determines the user's access level and accepts only three values: admin, supplier, or distributor. There are no foreign key relationships as the system manages a single entity type.



### 3.2 Data Access Mechanism

The application interacts with the database exclusively through the Flask-SQLAlchemy ORM. No raw SQL statements are written anywhere in the application code. The ORM translates Python method calls into the appropriate SQL statements internally and manages the database connection lifecycle automatically.

The following data access patterns are used throughout the application. During registration, the system performs two SELECT queries to check username and email uniqueness, followed by an INSERT to create the new user record. During login, the system performs a SELECT query to retrieve the user record by username. During session restoration on each request, the system performs a SELECT by primary key to reload the user object. During admin panel access, the system performs a SELECT of all records. During user deletion, the system performs a SELECT by primary key followed by a DELETE. All write operations -INSERT and DELETE are wrapped in SQLAlchemy's session transaction model and committed explicitly, ensuring atomicity.

### 3.3 Data Retention Policies

User records are retained in the SQLite database indefinitely until explicitly removed by an admin user through the admin panel's delete functionality. The system does not implement any automated data expiry, time-based record removal, soft delete mechanism (marking records as inactive without physically removing them), or data archival policy. When a user record is deleted through the admin panel, it is permanently and immediately removed from the database with no recovery mechanism. The database.db file itself persists on the local filesystem for the entire lifetime of the application installation and is not automatically backed up or rotated in the current version.

### 3.4 Data Migration

The database schema is created automatically when the application starts for the first time through SQLAlchemy's `db.create_all()` function, which reads the User model class definition and generates the corresponding SQL CREATE TABLE statement. No formal database migration framework is implemented in the current version of the application. If the User model is modified in a future version - for example by adding a `created_at` timestamp column, a phone number field, or a new role type - the existing database file would not automatically reflect the change. In such cases, the database file would need to be deleted and recreated from scratch, losing all existing user data, or a proper migration tool such as Flask-Migrate (which wraps the Alembic migration library) would need to be introduced to apply incremental and reversible schema changes without data loss.

## 4. Interfaces

The system provides four web-based user interfaces, each serving a distinct purpose in the user journey:

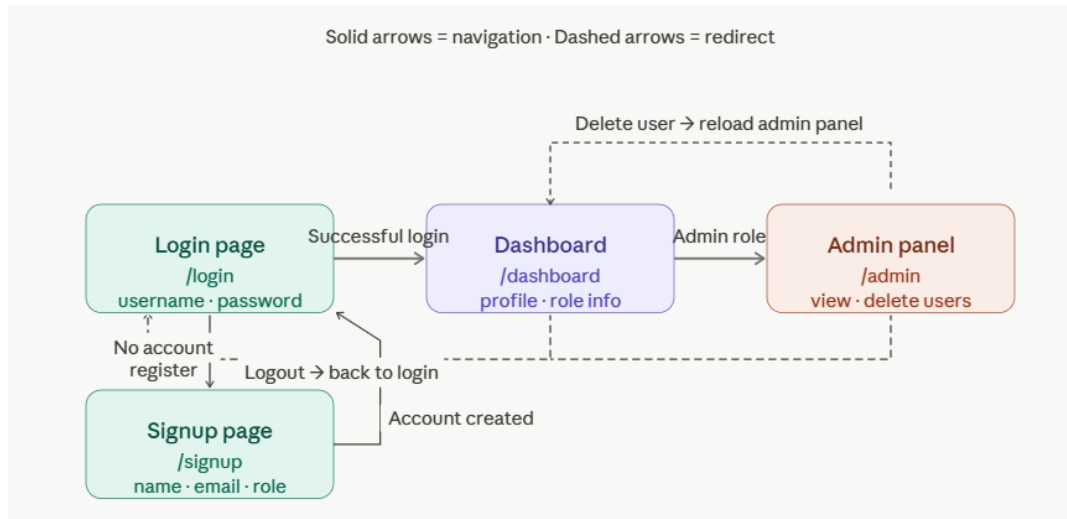
**Registration Page (signup.html)** — The first point of contact for new users at /signup. It presents a form where users enter their first name, last name, email, username, and password, and select their role from three options presented as styled interactive radio button cards. Validation errors are displayed as flash messages above the form.

**Login Page (login.html)** — The authentication gateway for returning users at /login. It presents a username and password form with error messages displayed on failed login attempts. Links at the bottom navigate to the signup page and a placeholder forgot password page.

**Dashboard (dashboard.html)** — The landing page after successful authentication at /dashboard. It displays the logged-in user's name, email, and role. It contains educational content explaining the difference between Authentication and Authorization, provides a logout link, and for admin users a visible link to the admin panel.

**Admin Panel (admin.html)** — An administrative interface accessible only to Admin users at /admin. It displays all registered users in a table showing ID, username, email, and role. Each row has a delete button pointing to /delete/{ { user.id } }. A self-deletion safeguard prevents the admin from removing their own account. A back button navigates to the dashboard.

All four interfaces share a consistent cybersecurity-themed visual design with a dark background, teal accent color, animated grid background, floating particle effects, and corner bracket decorations that reinforce the security-focused identity of the application.



## 5. State and Session Management

Flask-Login manages all user session state using signed cookies. The session lifecycle works as follows.

When `login_user()` is called after a successful login, Flask-Login stores the user's ID in the Flask session dictionary. Flask then signs this data using HMAC with the `SECRET_KEY` and stores it as a cookie in the user's browser.

On every subsequent request, Flask reads and verifies the session cookie. Flask-Login's `user_loader` callback is called with the user ID from the session and returns the full User object from the database, making it available as `current_user` for the duration of that request.

When a route decorated with `@login_required` is accessed without a valid session, Flask-Login redirects the user to the login page, preserving the originally requested URL as the next query parameter so the user can be returned there after login.

When `logout_user()` is called on logout, the session is cleared and the cookie is invalidated. The user is treated as unauthenticated on any further request and redirected to login if they try to access a protected route.

## 6. Caching

No caching mechanism is implemented in the current version of the application. Every HTTP request results in fresh database queries being executed. The user object is reloaded from the database on every request through the `user_loader` callback, and the admin panel executes a full read of all user records on every page load. There is no in-memory store, query result cache, or HTTP response cache configured at any layer of the application.

This is intentional and appropriate for the current scope of the project. In a future production deployment with a larger user base and concurrent sessions, Flask-Caching with an in-memory backend such as Redis or Memcached could be introduced to cache frequently accessed data and reduce repeated database reads under load.

## 7. Non-Functional Requirements

### 7.1 Security Aspects:

- **Password security** — User passwords are never stored in plain text. Werkzeug applies the PBKDF2-HMAC-SHA256 hashing algorithm with a randomly generated salt before storing passwords in the database. This means that even if the database file is accessed directly, no usable plain text passwords can be recovered from the stored hashes.
- **Session security** — Sessions are maintained through cookies signed using HMAC with the application's `SECRET_KEY`. The cookie contents are base64 encoded but any modification causes Flask to reject the session entirely. In a production deployment the `Secure` and `HttpOnly` cookie flags should be enabled and HTTPS should be used.
- **Two-layer access control** — The first layer enforced by `@login_required` ensures only authenticated users access protected pages. The second layer enforced by role checks inside admin route handlers ensures only Admin users

access administrative functionality. Both layers operate entirely server-side and cannot be bypassed through client manipulation.

- **Input validation** — All form data is validated server-side before any database operation is performed. Required fields, minimum length constraints, role validity, and uniqueness of username and email are all checked before a user record is created.
- **Username enumeration prevention** — The login route returns an identical error message for both an unknown username and a wrong password, preventing attackers from using the login form to determine which usernames are registered.

## 7.2 Performance Aspects:

The application is designed for local development and academic demonstration rather than high-traffic production use. SQLite is a serverless file-based database that provides sufficient performance for single-user and low-concurrency scenarios. The UNIQUE constraints on username and email columns cause SQLite to create implicit indexes on these fields, meaning login queries which filter by username execute as efficient indexed lookups and remain fast regardless of how many users are registered.

The admin panel uses a query that retrieves all user records at once. For a large user base this would need pagination. The Flask development server is single-threaded and suitable for testing only. For production, the application should be served by a multi-worker WSGI server such as Gunicorn. The architecture supports a straightforward scaling path SQLite can be replaced with PostgreSQL by changing a single configuration line since all database access goes through SQLAlchemy, leaving all application logic unchanged.



## 8. References

- Flask Official Documentation: <https://flask.palletsprojects.com>
- Flask-Login Documentation: <https://flask-login.readthedocs.io>
- Flask-SQLAlchemy Documentation: <https://flask-sqlalchemy.palletsprojects.com>
- Werkzeug Documentation: <https://werkzeug.palletsprojects.com>
- SQLite Official Documentation: <https://www.sqlite.org/docs.html>
- Python 3 Official Documentation: <https://docs.python.org/3/>
- OWASP Authentication Cheat Sheet:  
[https://cheatsheetseries.owasp.org/cheatsheets/Authentication\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html)
- OWASP Top Ten Web Security Risks: <https://owasp.org/www-project-top-ten/>